INTERVIEW PREPARATION · BAIREPORBO

How BairePorbo Actually Works

A walk through every subsystem you might be asked about — the mechanism, the reason it was built that way, and the number or trade-off that proves you understand it.

1.76K Android installs · 4.7★	**4,135** Web visitors / 3 mo	**375** Tests, 18 files
33K Lines, web `src/`	**42** API route handlers	**254** Commits since May

01 The sixty-second pitch

Open with this. It buys you the framing for every question that follows.

> SAY ROUGHLY THIS
>
> "BairePorbo is a production scholarship-discovery platform for Bangladeshi students — a Next.js web app and an Expo Android app sharing one API. About 1,700 installs and 4,000 web visitors a quarter, all organic.
>
> The core of it is a retrieval-augmented AI mentor. Students ask questions in Bengali or English and get answers grounded in our own scholarship database rather than the model's memory. The engineering problem I care most about there is **factual accuracy on deadlines** — a model that invents a deadline does real damage to a student. So retrieval decides *which* scholarships are relevant, but every fact the model is allowed to quote is re-read from the live Postgres row at request time."

That last sentence is your strongest single technical claim. It shows you understand what RAG is actually for, and that you found a failure mode most people never think about. Expect them to pull on it — section 03 is the detail.

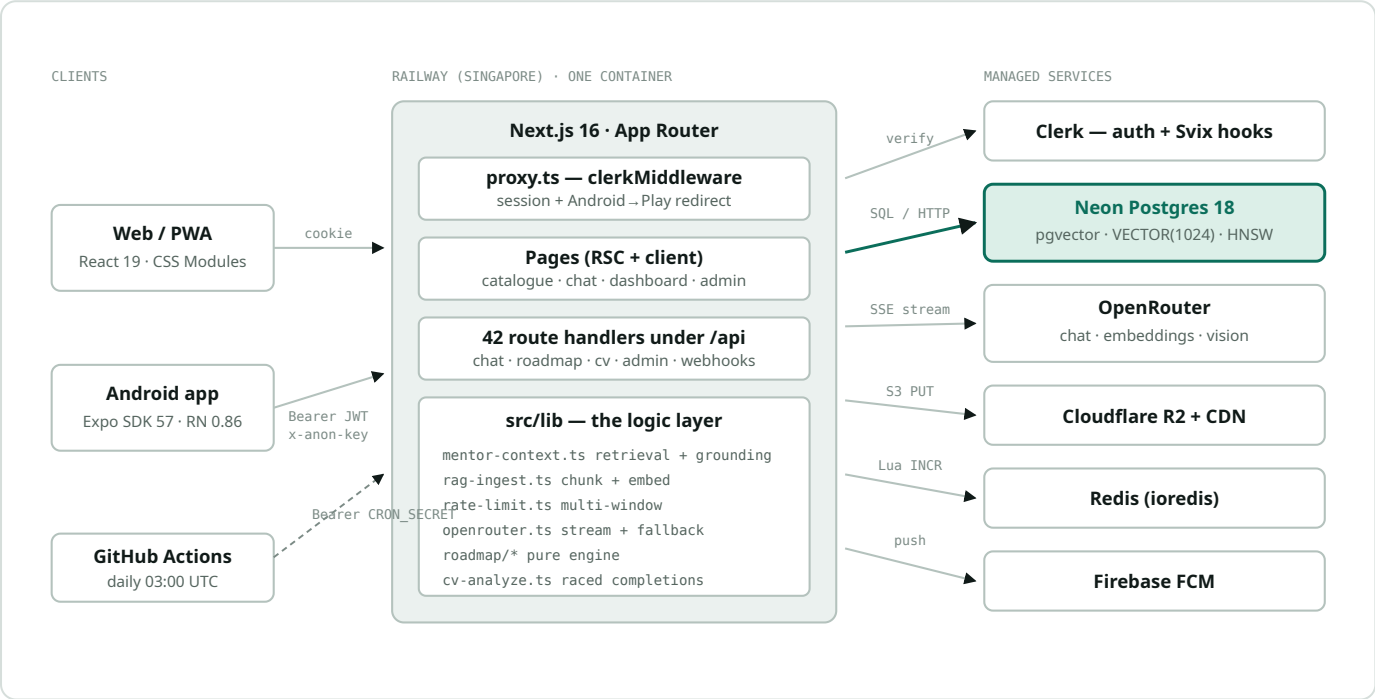
WHAT IT DOES

- **AI mentor chat** — streaming, RAG-grounded, profile-personalised, with a 3-message anonymous trial.
- **Scholarship catalogue** — filterable, AI-enriched, SEO-slugged.
- **Semantic matching** — vector similarity between a student profile and scholarship content.
- **Study-abroad roadmap** — a deterministic readiness engine over six weighted pillars.

- **Academic CV builder + AI analyser** — upload a PDF/DOCX, get structured critique.
- **Admin pipeline** — paste a URL, AI parses and enriches it, publish, auto-ingest to the vector store.
- **Bilingual UI + push notifications** — Bengali/English, FCM deadline reminders.

02 System architecture

One Next.js 16 application on Railway serves both the website and the API. The Android app is a separate Expo client that calls the exact same endpoints — there is no second backend. Everything else is a managed service.



One backend, two clients. The Android app is not a wrapper — it is a native Expo client hitting the same production API with a Clerk Bearer token instead of a cookie. The shared `packages/shared` package holds the TypeScript wire types and a typed fetch client, so a response-shape change breaks the mobile build at compile time rather than at runtime.

> **LIKELY QUESTION**
>
> *"Why one Next.js app instead of a separate backend service?"* — One deployable, one auth context, zero network hop between page and data, and the App Router's route handlers are a perfectly good REST layer. There *is* an empty NestJS scaffold at `apps/api` from an early plan; it was never used, and cutting it was the right call for a solo-maintained product. Say that plainly — an abandoned scaffold you can justify beats pretending it isn't there.

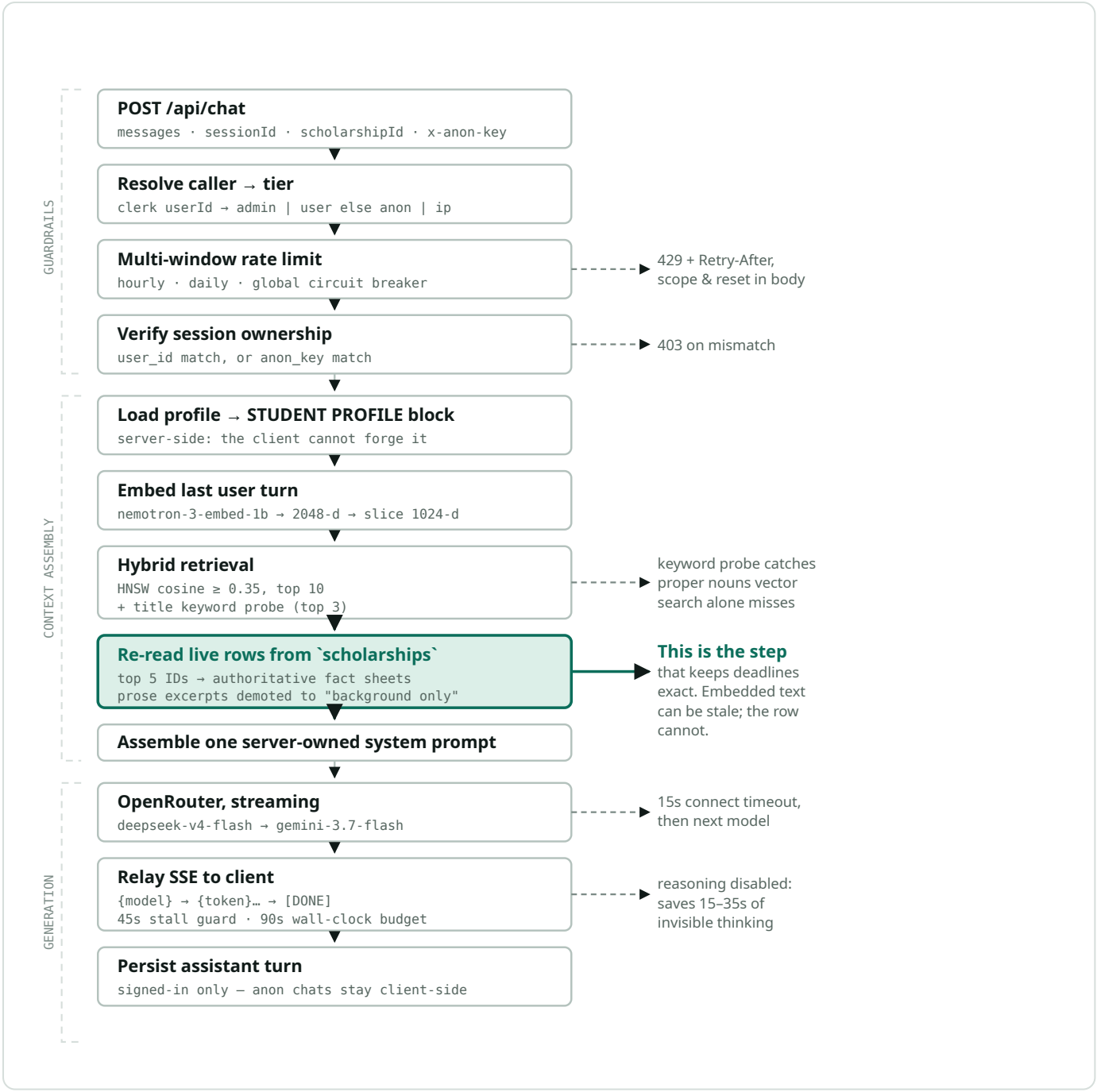
03 The RAG mentor — your flagship

This is the subsystem worth the most interview time. Know it cold.

The problem it solves

A student asks "when is the Erasmus Mundus deadline?" A raw LLM answers from parametric memory — it will confidently produce a plausible date that is months wrong. Naïve RAG improves this but does not fix it: you embed the scholarship description, retrieve a chunk, and the model quotes whatever date was in the *text you embedded*. That text is a snapshot. When an admin updates the deadline in the database, the embedded copy is stale until someone re-ingests.

> **THE DESIGN DECISION**
>
> Vector search is used **only to decide which scholarships are relevant**. The facts the model is permitted to quote are then re-read from the live `scholarships` table in the same request. Retrieved prose rides along as background and is explicitly ranked below the fact sheets in the prompt.

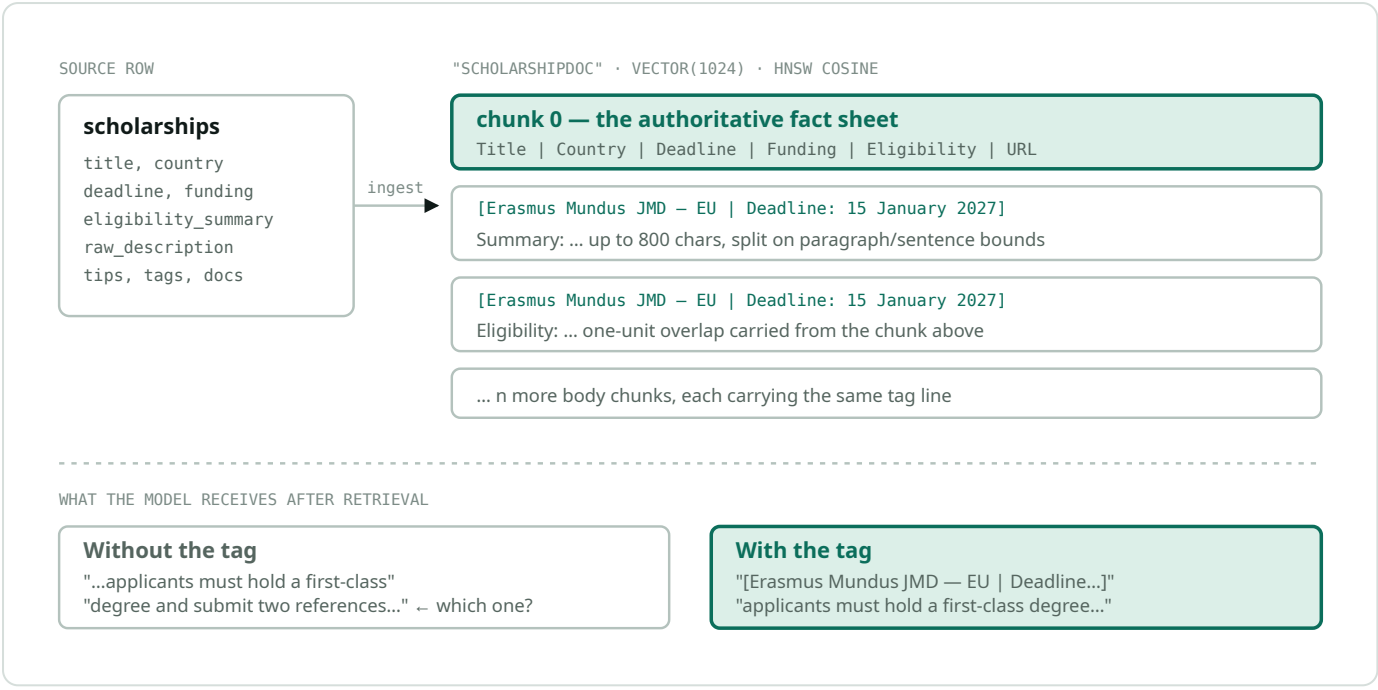


One chat request, end to end. The two highlighted steps are the grounding mechanism: retrieval yields IDs, the database yields facts. Everything above them is a guard, everything below is delivery.

Chunking: why every chunk carries its own identity

This is the second thing to know cold, because it is a bug you found and fixed rather than a pattern you copied.

Naïve chunking splits a description into 800-character pieces. Retrieval then returns a fragment from the middle of one — "applicants must hold a first-class degree and submit two references by…" — and the model has *no idea which scholarship it belongs to*. With five scholarships in the prompt it will attach that sentence to the wrong one.



Chunk 0 answers the question; every other chunk says who it is. Making the fact sheet its own chunk means "when is the deadline for X" can match a chunk that literally contains the answer, rather than hoping a prose fragment mentions it.

THE CHUNKER ITSELF

- Splits on paragraph breaks first, then sentence boundaries, so a chunk never opens mid-word.
- A single sentence over 800 chars is hard-cut as a last resort.
- Packing carries the trailing unit forward into the next chunk — **one-unit overlap**, so a fact spanning a boundary survives.
- Re-ingest embeds everything first, then deletes the old rows. **A failed ingest leaves the scholarship searchable with its previous vectors** rather than wiping it.

Embeddings: 2048 → 1024

The model is `nvidia/nemotron-3-embed-1b:free` via OpenRouter, which only emits its native 2048-dim vector. The column is `VECTOR(1024)`. So the code takes the first 1024 dimensions and **re-normalises to unit length**:

```
const sliced = embedding.slice(0, 1024);
const mag = Math.sqrt(sliced.reduce((s, x) => s + x * x, 0));
return sliced.map((x) => x / mag);   // L2 re-normalize
```

KNOW WHY THE RE-NORMALISATION MATTERS

The index is `vector_cosine_ops`. Cosine similarity divides by both magnitudes, so truncation alone would *technically* still work — but re-normalising to unit length makes cosine distance equivalent to a dot product, keeps every stored vector on the same scale, and means the similarity values are comparable across rows. It relies on the embedding being **Matryoshka-style**: trained so that a leading slice is itself a usable representation. If they ask "doesn't slicing lose information?" — yes, some, and that's the accepted trade for halving index size and memory. The alternative was re-training or a different model.

THE THRESHOLD STORY — A GOOD ONE TO VOLUNTEER

The match threshold is `0.35`, configurable via `RAG_MATCH_THRESHOLD`. It used to be `0.7`, inherited from the earlier `e5` embedding model. After switching models, **0.7 discarded almost every match** — retrieval silently returned nothing and the mentor quietly fell back to answering from parametric memory. The symptom was not an error; it was slightly-wrong answers. It was caught by logging `rag.context.empty` and noticing how often it fired.

The lesson to state out loud: *a similarity threshold is a property of the model, not of the application — it must be re-tuned whenever the embedding model changes, and silent-empty-retrieval needs its own log line because it does not throw.*

RETRIEVAL CONSTANTS

CONSTANT	VALUE	WHY
`RAG_MATCH_THRESHOLD`	0.35	Tuned for nemotron; 0.7 was empty
`RAG_MATCH_COUNT`	10	Vector hits fetched
`RAG_MAX_SCHOLARSHIPS`	5	Fact sheets in the prompt
`MAX_EXCERPTS`	5	Prose fragments, background only
`CHUNK_BODY_CHARS`	800	Body budget before the tag prefix
`MAX_HISTORY`	12	Turns of conversation sent upstream
`MAX_OUTPUT_TOKENS`	4096	Bengali costs 2–4× English tokens

DETAIL WORTH DROPPING

The 4096 output budget is not arbitrary. **Bengali text costs roughly 2–4× the tokens of the same content in English** under these tokenizers, so a budget that looks generous in English truncates a Bengali answer mid-sentence. That's the kind of constraint you only find by running a product in a non-Latin script.

Prompt ownership

The system prompt is assembled server-side and is never replaceable by the client. The scholarship detail page *does* send page context as a `system` message — those are stripped out of the message list and folded in under a labelled `PAGE CONTEXT PROVIDED BY THE APP` heading. Two reasons: security (a client could otherwise overwrite the grounding rules), and behaviour — **two competing system messages made the model pick one arbitrarily**.

The prompt also injects today's date in `Asia/Dhaka` so the model can judge whether a deadline has passed, and the student's profile so it never asks for a CGPA it was already given.

04 Streaming and failure handling

The route returns a `ReadableStream` with `Content-Type: text/event-stream`. It does not pipe OpenRouter's bytes through — it parses each SSE frame, extracts `choices[0].delta.content`, and re-emits its own simpler protocol.

```
data: {"model":"deepseek/deepseek-v4-flash"}   ← first, always
data: {"token":"Erasmus"}
data: {"token":" Mundus"}
data: [DONE]
```

Re-emitting rather than proxying buys three things: the client learns which model actually answered (useful when a fallback fired), the full text accumulates server-side for persistence, and the timeout guards can inject a message into the stream.

Three independent timeouts

GUARD	VALUE	PROTECTS AGAINST
`CONNECT_TIMEOUT_MS`	15s	Model never starts responding → fail over to the next one. Cleared the moment `fetch()` resolves, so it can never cut a live stream.
`STREAM_STALL_MS`	45s	Connection open but no token arriving. Resets on every chunk.
`CF_ORIGIN_BUDGET_MS`	90s	Total wall clock from request start. Cloudflare's origin proxy cuts at 100s — this stops first so the user gets a clean message instead of a hung stream.

> **WHY 45S AND NOT 10S**
>
> This is a great "I measured it" answer. Chat does not disable reasoning at the fetch level in the same way CV analysis does, and these models can spend **15–35 seconds emitting invisible reasoning tokens** before the first visible character. A 10-second stall guard would fire constantly on perfectly healthy replies. The guard has to sit safely above the observed worst case.

When a guard fires, the stream is not just closed. The partial answer is kept and a sentence is appended — *"The mentor's response was cut short"* — so the student sees an explanation rather than a reply that stops mid-word.

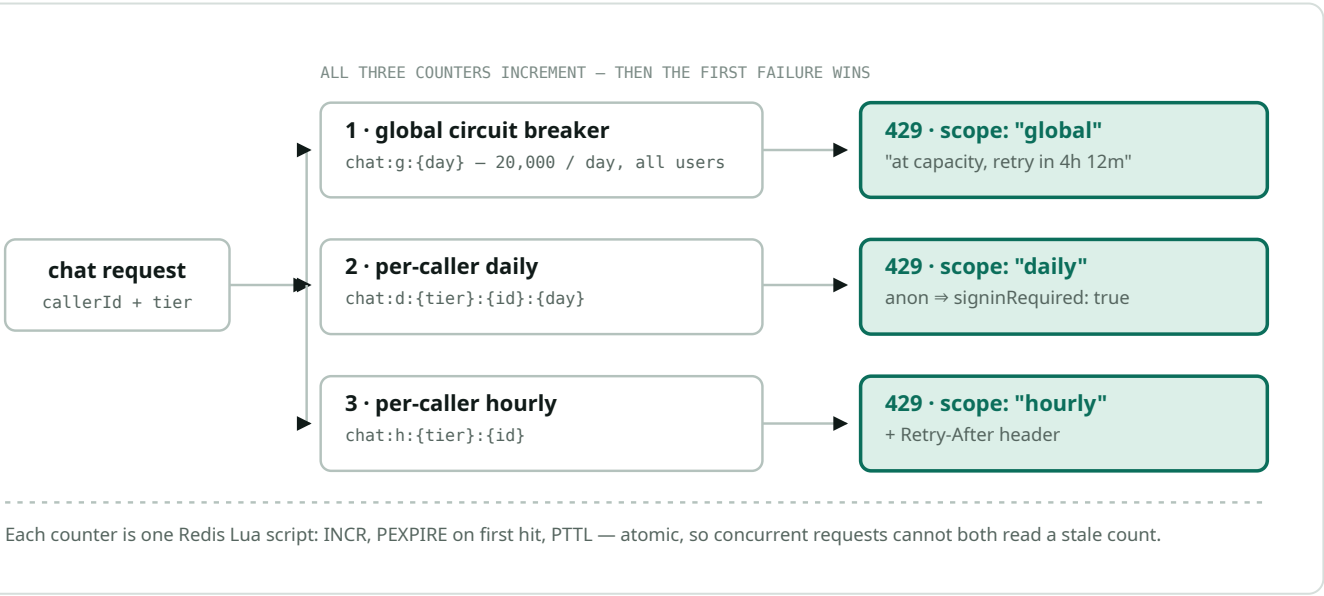
Model fallback chain

- **Text:** `deepseek/deepseek-v4-flash` → `google/gemini-3.7-flash`. Chosen partly because they run on *different infrastructure* — a DeepSeek outage does not take chat down.
- **Attachments (image/PDF):** a single vision model, `openai/gpt-5.6-luna`. DeepSeek is text-only, and Gemini vision tokens are expensive enough to burn the free-tier budget.
- **PDFs** pin the `cloudflare-ai` parser engine explicitly, because OpenRouter's default engine can be paid OCR.

> **THE LATENCY DECISION**
>
> Chat passes `reasoning: { enabled: false }`. The comment in the code is the answer to give: a mentor Q&A does not need multi-step reasoning the way structured CV extraction does, and disabling it removed **the single biggest lever on perceived chat latency**. Cost control and UX aligned here — that's worth saying, because it shows you weighed quality against responsiveness rather than just picking the cheapest option.

05 Rate limiting

The product runs on free-tier model quotas with anonymous access. Without limits, one script drains the day's budget for every real student. So the limiter is a genuine business requirement, not boilerplate.



Ordered windows, one decision. Global is checked first so an abuse spike is visible before any individual user's limit would have fired. The 429 body carries `scope`, `resetMs`, a human `resetIn` string and remaining counts, so the client can render the right message without guessing.

THE TIERS

TIER	CHAT / HR	CHAT / DAY	ATTACHMENTS / DAY
Anonymous	3	3	0 — sign-in required
Signed-in student	6	15	2
Admin	50	200	20
Global breaker	—	20,000	300

> TWO DESIGN POINTS TO RAISE UNPROMPTED
>
> **Attachments have a completely separate quota.** Vision tokens cost far more than text tokens, so without a second set of counters a student's 15 daily messages could all become 15 vision calls. Separating them means running out of document-reading does *not* stop you typing questions — and the error copy says exactly that: *"You can keep typing questions — document reading is limited to 2 per day."* Degrading one capability instead of blocking the feature is a deliberate UX choice.
>
> **Hourly equals daily for attachments (2 and 2).** That looks redundant until you see why: it lets a student send two screenshots in one sitting, but stops someone using the UTC-midnight reset to fire four in two minutes.

REDIS, AND WHAT HAPPENS WITHOUT IT

Each window is one Lua script evaluated server-side: `INCR`, then `PEXPIRE` only if the counter just became 1, then read `PTTL`. Atomicity matters — a read-then-write in application code lets two concurrent requests both see the old count.

There is an in-memory `Map` fallback when `REDIS_URL` is unset. **Be honest about its limits:** it is per-process, so it is correct for local development and for a single container, and it would under-count the moment the app scaled horizontally. Redis is what makes the limit real in production.

IDENTIFYING AN ANONYMOUS CALLER

Priority is `cf-connecting-ip`, then `x-forwarded-for`, then `x-real-ip`. Cloudflare's header is checked first because Cloudflare **overwrites it with the true client IP and strips any client-supplied copy** — it is both more accurate and harder to spoof. Falling back to a proxy IP would drop every visitor into one bucket. Above IP, a client-generated `x-anon-key` gives a stable per-install identity on mobile.

06 Authentication and identity

Clerk handles credentials; Postgres holds the application role. That split is the whole design.

HOW IT FITS TOGETHER

- `clerkMiddleware` in `src/proxy.ts` runs on every non-static route.

- Web uses cookie sessions; the Android app uses `@clerk/clerk-expo` and sends a **Bearer session token** to the same endpoints.
- `profiles.role` is `student` or `admin`, stored in *our* database, not in the Clerk JWT.
- Every admin route calls `requireAdmin()` server-side, which re-reads the role from Postgres on each request.
- Client-side `admin-guard.tsx` hides admin UI — presentation only, never the security boundary.

> **EXPECT: "WHY ISN'T THE ROLE IN THE JWT?"**
>
> Putting it in the token would save a query per admin request. The trade-off is **revocation latency**: a JWT claim stays valid until the token expires, so demoting an admin would not take effect immediately. A database read is one indexed primary-key lookup against a serverless Postgres in the same region — cheap enough that instant revocation is worth it. Also say the honest version: admin traffic is a tiny fraction of requests, so this is not a hot path worth optimising.

The webhook

Clerk fires `user.created` to `/api/webhooks/clerk`. The handler:

1. Reads the **raw body before** `.json()` — signature verification is over the exact bytes.
2. Verifies the Svix signature with `CLERK_WEBHOOK_SECRET`; a bad signature is a 401.
3. Inserts the `profiles` row with `ON CONFLICT DO NOTHING` — webhooks can be delivered more than once, so the handler must be idempotent.
4. Seeds three onboarding tasks so a new dashboard is never empty.

> **COMPLIANCE DETAIL WORTH MENTIONING**
>
> The same endpoint handles `user.deleted` by purging chat messages, sessions, bookmarks, tasks, CVs, analyses and the profile row. That exists because **Google Play's user-data policy requires an in-app account-deletion path** for a published app. It's a nice example of a platform requirement driving a backend design.

The Postgres story you should be able to tell

The database started on Supabase, where authorisation was enforced by **row-level security policies** keyed to `auth.uid()`. Migrating to Clerk + Neon meant RLS could no longer see the user — **authorisation moved into the application layer**, into `requireAdmin()`, explicit `WHERE user_id = …` clauses, and ownership checks like the one the chat route does on `chat_sessions`.

Be straight about the trade: RLS is a stronger guarantee because the database enforces it even if a query is written carelessly. Application-layer checks are more flexible and easier to test, but they are only as good as the discipline applied to every route. That is a mature thing to say, and it is exactly the follow-up a good interviewer will ask.

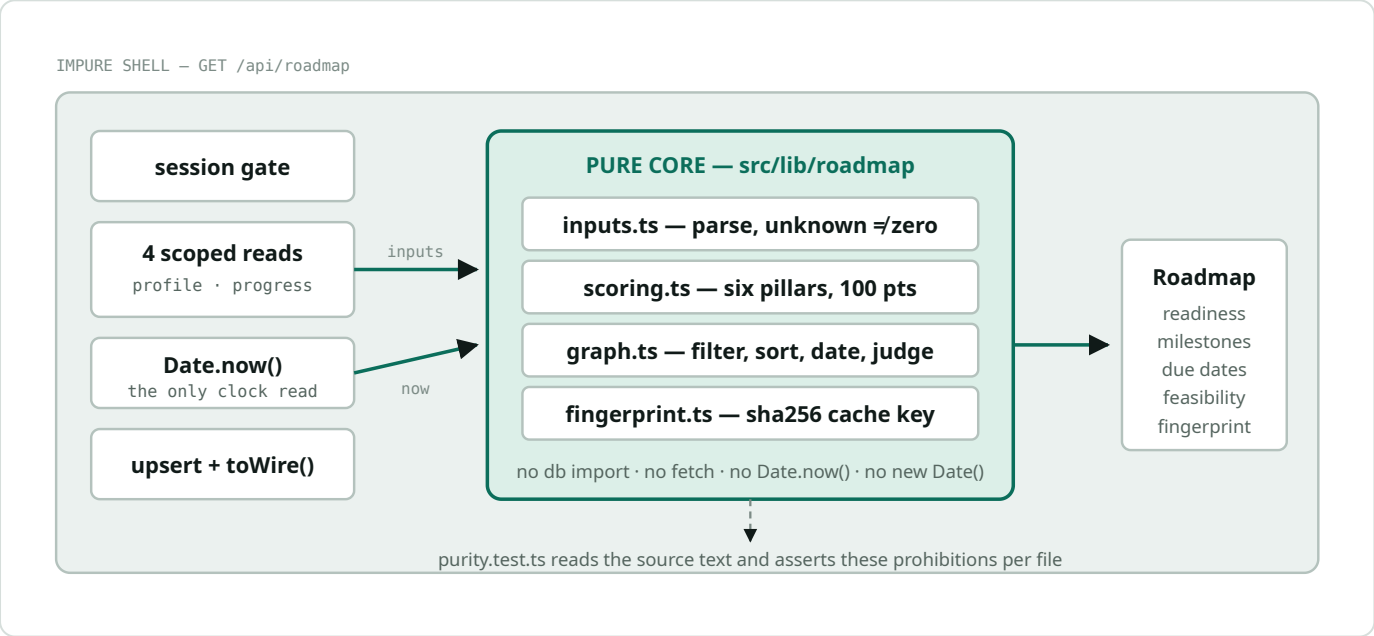
One artefact survives from that era: `match_scholarship_docs` is still `SECURITY DEFINER`. Originally that was to bypass RLS under a user JWT. Know why it was there, and that it is now vestigial.

07 The roadmap engine

If the interviewer is a senior engineer, **this is the subsystem that will impress them most** — more than the AI work, because it shows software-design judgement rather than API integration. It is a pure, deterministic scoring engine with its purity mechanically enforced by tests.

What it computes

A student's readiness score out of 100, across six weighted pillars, plus an ordered milestone graph with due dates and feasibility. **No AI call happens on the read path.** A roadmap is arithmetic over the student's own stored data; the AI narration is generated separately and cached.



Functional core, imperative shell. Every side effect lives in the route; the engine is a function from data to data. That is what makes a readiness score reproducible from a fixture and testable without mocking a database.

Three rules enforced by structure, not vigilance

1 · A STATUS WRITE CANNOT MOVE THE SCORE

`scoreProfile` takes `RoadmapInputs` and nothing else, and `RoadmapInputs` contains no field derived from the milestone-progress table. So a student cannot inflate their readiness by ticking milestones off. **The anti-gaming rule is a consequence of the function signature, not a check someone could forget to write.** That sentence is worth memorising — it is the single best line in this codebase.

2 · UNKNOWN IS NOT ZERO

Every parser returns `T | null`. A declared zero — "none", "0", "fresher" — is a known zero and scores 0. Unparseable prose returns `null`, which marks the pillar `known: false`: it earns 0 points but is

barred from producing a "weakness", and costs Confidence instead. A student who wrote "some conference work" has not told us they have no papers, so the engine must not accuse them of having none.

3 · BELOW THE GATE, NO NUMBER AT ALL

Readiness is withheld entirely unless degree and CGPA are known and Confidence reaches 40 — meaning at least four of the eight student-supplied inputs are readable. *A profile the engine cannot score is a profile it must not diagnose.*

THE WEIGHT TABLES

PILLAR	BACHELOR	MASTER	PHD
Academics	25	20	15
English	25	20	15
Documents	25	25	20
Research	5	15	**30**
Experience	10	10	10
Application progress	10	10	10
TOTAL	100	100	100

Research is 5 points for a bachelor's applicant and 30 for a PhD applicant — the domain knowledge is encoded in the weights. A test asserts every column sums to exactly 100, because **a column that silently drifted to 99 would cap every student in that bracket at 99% ready**.

The fingerprint

The `roadmaps` table is a cache, not a source of truth — everything on the row except the AI narration is recomputable. Freshness is decided by a sha256 over `${ENGINE_VERSION}:${stableStringify(inputs)}`:

- **Key-sorted** serialisation, so building the inputs object in a different field order does not invalidate every student's narration.
- **Arrays keep their order** — a list's order is data.
- `undefined` members are dropped, so `{}` and `{ lor_count: undefined }` hash alike.
- **Prefixing `ENGINE_VERSION`** means bumping the version invalidates every cached narration on purpose — the deploy-time cache-busting mechanism.

Testing: 375 tests, and what kind

Don't just quote the number — say what the tests *do*:

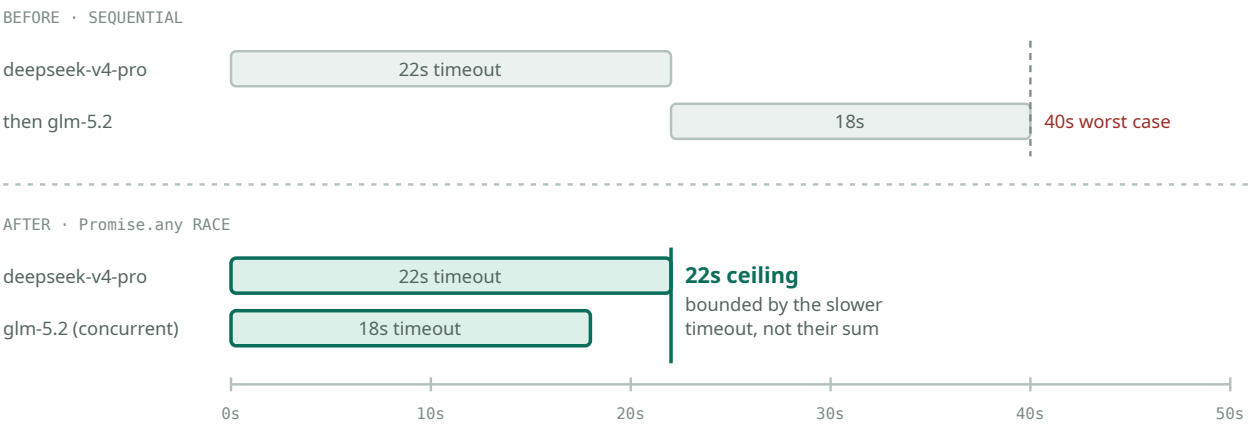
- **Property-based tests with `fast-check`.** Instead of asserting fixed inputs, they generate thousands of random profiles and assert invariants: readiness never exceeds 100, a known input never lowers a score, pillar shares stay in [0,1].
- **A source-text purity test.** `purity.test.ts` strips comments from each engine file and greps the executable text for `fetch(`, `Date.now()`, `new Date()` with no argument, and any database import. It is a lint rule expressed as a test, and it fails the build the moment someone reaches for the clock inside the engine.
- **Persona fixtures.** Hand-computed expected scores for realistic student profiles — the check that the weights produce sensible human outcomes, not just internally consistent ones.

> **IF ASKED "WHY PROPERTY TESTS HERE AND NOT ELSEWHERE?"**
>
> Because this is the only part of the system that is *pure and total* — a deterministic function over a well-defined input domain. Property testing pays off exactly there. An API route that talks to Postgres and OpenRouter has no such domain to quantify over; integration tests are the right tool for those. Matching the testing strategy to the shape of the code is the point.

08 CV analysis — racing two models

A small, sharp piece of latency engineering. A student uploads a CV; text is extracted (`unpdf` for PDF, `mammoth` for DOCX) and sent to an LLM for structured critique returned as JSON.

The best model for the task, `deepseek-v4-pro`, has **erratic latency — measured runs on the same prompt ranged from 26s to over 40s**. Sequential fallback meant a slow spell cost 22s of waiting plus another 18s before the student saw anything.



The wait becomes the max, not the sum. First usable JSON wins. Only if *both* reject does a third attempt fire — `gpt-4o-mini` at 16s, keeping the absolute worst case near 38s, inside the route's 60s `maxDuration`.

> **THE SUBTLE DECISION — EXPECT TO BE ASKED**
>
> *"Why race against glm-5.2 rather than the cheaper deepseek-v4-flash?"*
>
> Because in a race, **the fastest response wins regardless of quality**. Flash is noticeably worse at this specific task — the prompt demands a lot of structured, opinionated critique, and flash-tier models either shortcut it or emit invalid JSON under that load. Racing it against pro would let a weaker answer win just by finishing first. `glm-5.2` is full-size and comparable in quality, so either winner is acceptable. And being **a different vendor entirely**, it fails independently of a DeepSeek-side outage — which is the failure mode actually observed in production.
>
> That is the whole art of a race: the candidates must be interchangeable in quality, or you have built a machine for picking the worst answer quickly.

TWO MORE DETAILS

- **`response_format: json_object`.** Provider-enforced valid JSON. Without it, models routinely emit *almost*-valid JSON — a trailing comma, an unescaped quote inside an array — which no amount of lenient client-side parsing reliably recovers. Parsing is still defensive (two strategies, then a logged preview of the raw content), but the constraint does the heavy lifting.
- **Reasoning disabled.** Not just for latency: reasoning tokens consume the `maxTokens` budget, which truncated the JSON mid-object. Two different bugs, one fix.
- **Output is normalised, not trusted.** `normalizeAnalysis()` clamps the score to 0–100, filters non-strings out of every array, and coerces an unrecognised section rating to "adequate". LLM output is untrusted input.

09 Data model

TABLE	HOLDS	WORTH KNOWING
`profiles`	Role + ~15 student fields	PK is the Clerk user ID — no separate internal ID
`scholarships`	Catalogue + AI-enriched fields	`deadline` is **free text** — see below
`"ScholarshipDoc"`	RAG chunks	`VECTOR(1024)`, HNSW cosine index
`chat_sessions` / `chat_messages`	Threads and turns	Session belongs to `user_id` *or* `anon_key`
`roadmaps` / `milestone_progress`	Cached roadmap + per-milestone status	Cache keyed by fingerprint, not a source of truth
`user_cvs` / `cv_analyses`	CV documents and AI critiques	
`user_bookmarks` / `user_tasks`	Saved scholarships, to-dos	Tasks seeded by the Clerk webhook
`guides`	CMS long-form content	Slugs, FAQs, cover images on R2
`push_tokens` / `push_reminders_sent`	FCM devices, dedupe ledger	Makes the daily cron idempotent

> **THE FREE-TEXT DEADLINE — OWN THIS ONE**
>
> `scholarships.deadline` is `TEXT`, not `DATE`, because real scholarships say "Rolling", "TBA", or "varies by programme". A `DATE` column cannot hold those, and forcing one would mean either losing information or inventing a date.
>
> **The cost is real and you should name it:** date filtering cannot happen in SQL. The push-digest cron loads every bookmarked scholarship with a non-null deadline and filters in JavaScript. At current scale that is trivial; at 10,000 scholarships it is not. **The fix I'd ship:** keep the text column for display and add a nullable `deadline_date DATE` generated at write time, indexed, used for filtering. Best of both — and proposing it unprompted is far stronger than defending the current design.

QUERY SAFETY

All queries go through Neon's tagged-template client, which parameterises interpolations — `sql`SELECT … WHERE id = ${userId}`` sends `$1`, not string concatenation. Dynamic queries (a PATCH with a variable SET clause) use `sqlQuery(text, params)` with positional placeholders. **No user input is ever concatenated into SQL.**

VECTOR INDEX

HNSW — a navigable small-world graph — rather than IVFFlat. HNSW has better recall at a given speed and, importantly, **does not need retraining as rows are inserted**; IVFFlat's centroids degrade as the data grows unless you rebuild. For a catalogue that gets new rows weekly, HNSW is the lower-maintenance choice. The trade-off is slower index build and more memory — irrelevant at this size.

10 Infrastructure and two migrations

Both migrations happened on a **live product with real users**. That is the part worth emphasising — anyone can pick a stack, fewer have moved one under traffic.

June 2026 — off Supabase

BEFORE	AFTER	WHY
Supabase Auth	Clerk	Native mobile SSO; Supabase's RN story was the blocker
Supabase Postgres	Neon	Serverless driver over HTTP, branching, independent of the auth vendor
Supabase Storage	Cloudflare R2	Zero egress fees; CDN on a custom domain

Moved: **132 users, 54 scholarships, 152 embedding documents, 82 chat sessions** — with zero downtime. The hard part was not the data; it was that **authorisation had to be rewritten**, since RLS policies keyed to `auth.uid()` stopped working once Clerk owned identity. Every policy became an application-layer check.

September 2026 — Vercel to Railway

The reason is concrete and worth stating precisely: **serverless CPU and duration limits were the wrong shape for this workload.** SSE chat streams, CV analysis and RAG ingest are long-running. Railway runs a persistent container in Singapore — close to users in Bangladesh — where a 90-second stream is unremarkable.

Knock-on changes: Vercel Cron became a GitHub Actions workflow posting to `/api/cron/push-digest` with a Bearer `CRON_SECRET`; Vercel Analytics became Cloudflare Web Analytics.

> **A SMALL DETAIL THAT READS AS PRODUCTION EXPERIENCE**
>
> The Redis client is constructed with `family: 0` (dual-stack). Railway's private networking is IPv6-only, while a local `REDIS_URL` is IPv4 — dual-stack is what makes one config work in both places. Tiny, but it is the kind of thing you only learn by deploying.

PUSH NOTIFICATIONS

A daily job at 03:00 UTC = 09:00 Dhaka. It reminds students at **7, 3 and 1 days** before a bookmarked scholarship's deadline. Two details:

- **Dhaka-local day maths.** Day counts are computed local-midnight to local-midnight with a fixed +6h offset, so "tomorrow" means tomorrow on the student's calendar, not on UTC's.
- **Idempotent by construction.** Every `(user, scholarship, milestone)` triple is recorded in `push_reminders_sent`, so a retry or a manual re-run cannot double-notify. Cron jobs get retried; a notification job that isn't idempotent will eventually spam someone.

SEO / I18N

English lives at unprefixed paths that are already indexed; Bengali lives under `/bn` and **only for pages whose main content is genuinely translated**. Scholarship and guide detail pages are deliberately *not* mirrored, because their body text is English in the database — duplicating them would create cross-locale duplicate content that suppresses site-wide quality signals. hreflang, the language toggle and the sitemap all read from one module, so they cannot drift apart.

11 The Android app

Expo SDK 57 / React Native 0.86, Expo Router for file-based navigation, NativeWind for styling, TanStack Query for server state. Live on Google Play at 4.7★.

THE SHARED CONTRACT

`packages/shared` holds the wire types and a framework-agnostic API client — no React, no Expo, no Node built-ins, just `fetch`. It centralises the base URL, Clerk Bearer injection, the `x-anon-key` header and error parsing. `ApiError` carries the parsed body, including a `signinRequired` getter so the anonymous-trial cap renders the right prompt.

The value: **changing an API response shape breaks the mobile typecheck** rather than shipping a runtime crash to the Play Store, where a rollback takes hours.

DETAILS WORTH HAVING READY

- **Token refresh:** `getToken({ skipCache: true })` after a 401, so an expired JWT is re-minted rather than replayed.
- **Streaming on RN:** the client takes an injectable `fetchImpl` because React Native's built-in fetch does not stream — the app passes `expo/fetch`.
- **The `www` host is mandatory** in production: the apex domain 301-redirects, and a 301 turns a POST into a GET, silently breaking every write.
- **Android homepage redirect:** middleware sends Android user-agents on `/` and `/bn` to the Play Store — but excludes crawlers so Googlebot still indexes the site, and honours `?web=1` as an escape hatch.

12 Question drills

Click each to reveal the answer. Practise saying them out loud — the goal is fluency, not recall.

RAG

Walk me through what happens when a student sends a chat message.

Eleven steps, and I'd narrate them as three phases. **Guardrails:** resolve the caller to a tier, check three rate-limit windows, verify session ownership if a session ID was sent. **Context assembly:** load the profile server-side and render it as a block, embed the last user turn, run vector search plus a title-keyword probe, take the top five scholarship IDs and re-read their live rows from Postgres as authoritative fact sheets. **Generation:** assemble one server-owned system prompt, stream from OpenRouter with a fallback model, relay tokens as SSE under a stall guard and a wall-clock budget, and persist the assistant turn for signed-in users.

The step I'd highlight is the re-read. Retrieval decides relevance; the database decides truth.

RAG

Why not just trust the embedded text? Isn't that what RAG is?

Embedded text is a snapshot taken at ingest time. When an admin corrects a deadline, every chunk containing the old date stays wrong until someone re-ingests — and the model will quote it confidently because it came from "the context". For most RAG applications a slightly stale paragraph is tolerable. Here the single most common question is "when is the deadline", and a wrong answer makes a student miss an application.

So retrieval does what vector search is genuinely good at — semantic relevance ranking — and the facts come from the live row. The prompt says explicitly that the verified block overrides anything the model remembers, that deadlines must be quoted exactly, that "typically" and "approximately" are forbidden about a stated value, and that if something isn't in the block it must be labelled general guidance to confirm on the official site.

RAG

Why add keyword search? Doesn't that undermine the point of embeddings?

It complements them. Embeddings capture meaning but are weak on **proper nouns**. A query like "Endeavour II deadline?" has almost no semantic content beyond a name — the embedding lands nowhere useful. A direct title match finds it instantly.

So the union is taken: vector hits first, ranked by similarity, then title-only matches appended, deduplicated, capped at five. This is standard hybrid retrieval, and it's cheap here because the title probe is a single SQL query.

I'd add the honest caveat: the probe uses `LIKE '%term%'`, which can't use a B-tree index. At 54 scholarships it's a sub-millisecond scan. At scale I'd move it to a GIN index over a `tsvector`, or Postgres trigram search.

EMBEDDINGS

You store 1024 dimensions from a 2048-dimension model. Explain.

The provider only emits the native 2048-d vector, but the column is `VECTOR(1024)`. I take the leading 1024 dimensions and re-normalise to unit length so cosine distance stays well-behaved and similarity values remain comparable across rows.

This works because the model is **Matryoshka-style** — trained so a leading slice is itself a usable representation, with information front-loaded into the early dimensions. You cannot do this with an arbitrary embedding model; truncating a non-Matryoshka vector discards meaningful dimensions.

The trade is some recall for half the index size and memory. At this corpus size the quality difference isn't measurable; at a much larger scale I'd benchmark full 2048 against the slice before deciding.

DEBUGGING

Tell me about a bug that was hard to find.

Two good ones.

The silent-retrieval bug. After switching embedding models, the match threshold stayed at 0.7 — tuned for the old model. The new model's similarity scores are distributed differently, so 0.7 rejected essentially everything. Nothing errored. Retrieval returned an empty set, the prompt had no verified-data block, and the model happily answered from memory. The answers were *plausible*, which is what made it hard: no exception, no 500, just subtly wrong output. I found it by logging `rag.context.empty` and noticing how often it fired. Threshold is now 0.35 and environment-configurable.

The lesson: a similarity threshold is a property of the model, not the application, and a fallback that silently degrades quality needs its own telemetry, because it never throws.

The anonymous-chunk bug. Retrieval returned mid-description fragments with no identifying information, so with five scholarships in the prompt the model attached eligibility rules to the wrong one. Fix: every chunk is prefixed with a title/country/deadline tag, and chunk 0 of each scholarship is a complete fact sheet. Both bugs had the same shape — no error, just degraded correctness.

DESIGN

How do you stop the AI hallucinating?

You don't stop it — you constrain what it's allowed to speak about, and structure the system so hallucination is bounded.

Four layers here: **(1)** facts come from live database rows, not from model memory or embedded snapshots. **(2)** The system prompt states explicit grounding rules — the verified block overrides memory, quote values exactly, never invent a deadline or URL, and if something isn't listed say so and label the rest as general guidance. **(3)** Dates are pre-formatted before the model sees them: an ISO date is rendered with its month spelled out alongside, because models rewrite `2026-06-05` into the wrong month far more often when they do the conversion themselves. **(4)** Today's date in Asia/Dhaka is injected so the model can judge whether a deadline has passed.

What I have *not* built is automated output verification — a post-generation check that every date in the answer appears in the context block. That's the obvious next layer.

SCALE

What breaks first at 100× the traffic?

In order:

- **The model budget.** The global circuit breaker at 20,000 chats/day is the real ceiling — it exists precisely because free-tier quotas are finite. That's a cost problem before it's an engineering one.
- **Embedding calls.** Every chat turn embeds the query; there's no cache. Common questions repeat heavily, so a hash-keyed embedding cache in Redis would cut a meaningful share of that latency and cost.
- **The title probe** — `LIKE '%term%'` is a sequential scan. Needs a trigram or full-text index.
- **The push-digest cron** loads all bookmarked scholarships and filters dates in JavaScript, because `deadline` is free text. Needs the generated `deadline_date` column so the filter moves into SQL.
- **Profile matching** fires one embedding call per preferred country, concurrently. Fine for three countries, wasteful for ten.

What wouldn't break: the HNSW index has enormous headroom, and Neon scales independently of the app.

ARCHITECTURE

Why does the roadmap engine avoid reading the clock?

Because a function that reads the clock can't be pinned by a fixture. If `scoreProfile` called `Date.now()`, the same student's score would differ between two test runs, and I'd have to mock the global clock to test anything.

So the engine is a pure function: inputs in, roadmap out. The route reads `Date.now()` once and passes it as a parameter. That's the functional-core / imperative-shell pattern — every side effect in the route, all logic in a testable core.

The part I'm most pleased with is that it's **enforced, not merely documented**. A test reads each engine file's source, strips comments, and asserts the executable text contains no `fetch(`, no `Date.now()`, no zero-argument `new Date()`, and no database import. The one allowed exception is `node:crypto` in the fingerprint module — hashing is computation, not I/O. If someone reaches for the clock inside the engine, the build fails and tells them why.

PRODUCT

A student games the system by ticking every milestone done. What happens?

Nothing. Their readiness score doesn't move.

`scoreProfile` takes `RoadmapInputs` and nothing else, and that type contains no field derived from the milestone-progress table. Stored progress reaches `graph.ts` and stops there — it decides which nodes render as done, never what the score is. So the anti-gaming rule is a consequence of the module boundary rather than a check someone could forget.

The score only moves when the underlying facts move: a real IELTS score entered, a document marked ready, a paper recorded.

TESTING

375 tests — what do they actually cover?

Concentrated where they pay off, not spread evenly for a coverage number. The bulk are on the roadmap engine, because it's the only pure, total part of the system.

Property-based tests with `fast-check` generate thousands of random profiles and assert invariants — readiness never exceeds 100, every weight column sums to exactly 100, an unknown pillar can never produce a weakness, adding a known input never lowers the score. The weight-sum test matters concretely: a column drifting to 99 would silently cap every student in that bracket at 99% ready, and no example-based test would catch it.

Persona fixtures hand-compute expected scores for realistic students, checking the weights produce sensible human outcomes rather than just internally consistent ones.

Then focused unit tests on the genuinely tricky pure helpers — embedding dimension fitting, chat attachment parsing, FCM error classification, i18n URL generation — plus route tests for the roadmap and profile endpoints.

The honest gap: **no end-to-end tests, and the chat route isn't directly tested.** It's the hardest thing to test — streaming, two external services, a database — and that's exactly why it deserves them. I'd start by extracting the SSE parsing loop into a pure function over a string iterator; that alone would make most of it testable without mocks.

ARCHITECTURE

Why Promise.any for CV analysis rather than a queue or a fallback chain?

A queue is the right answer for work the user isn't waiting on. Here they *are* waiting — they just uploaded a CV and are staring at a spinner — so the objective is bounded wall-clock latency, not throughput.

Sequential fallback made the worst case the *sum* of both timeouts, about 40 seconds. Racing makes it the *max*, about 22. Cost roughly doubles per analysis, but CV analysis is low-volume and user-initiated, so I traded tokens for latency deliberately.

The non-obvious constraint is that racing only works if the candidates are comparable in quality — otherwise you've built a machine for picking the worst answer fastest. That's why the partner is `glm-5.2`, a full-size model from a different vendor, rather than a cheaper flash-tier one. Different vendor also means independent failure, which is the outage mode I actually saw.

SECURITY

How do you stop someone abusing the anonymous chat?

Layered, because no single identifier is reliable for anonymous users.

Callers are keyed by `anon:{x-anon-key}` when the client supplies a stable per-install key, otherwise by IP. The IP comes from `cf-connecting-ip` first, because Cloudflare overwrites that header with the true client IP and strips any client-supplied copy — it's both more accurate and harder to spoof than `x-forwarded-for`.

Anonymous users get 3 messages per hour and per day, and zero attachments — vision tokens are expensive and only signed-in users get them. Above all of that sits a global daily circuit breaker at 20,000, checked first so an abuse spike is visible before any individual limit fires.

The honest limitation: an `x-anon-key` is client-generated, so it can be rotated. The IP limit is the real floor, and it can be worked around with enough IPs. For a free trial that's an acceptable bar — I'm protecting a budget, not a vault, and the global breaker is what actually caps the blast radius.

SECURITY

How is the admin area protected?

Server-side, on every request. Each admin route calls `requireAdmin()`, which resolves the Clerk user then reads `profiles.role` from Postgres and returns null unless it's `admin`. The client-side `admin-guard` component only hides UI — it's presentation, never the boundary.

The role lives in our database rather than the Clerk JWT specifically so revocation is immediate; a JWT claim would stay valid until expiry. Admin AI endpoints have their own rate limits on top — parse is 15 per 10 minutes per admin — because those hit paid models and a runaway loop would be expensive.

OPS

The AI provider goes down. What does a user see?

Depends where it fails, and each case is handled distinctly.

Primary model unreachable: a 15-second connect timeout fires and the request fails over to Gemini Flash — deliberately on different infrastructure. The user sees a normal answer; the first SSE event tells the client which model actually served it.

All models fail: 502 with the error, and the UI shows a retry.

Stream dies mid-answer: the stall guard fires after 45 seconds of silence, and rather than closing silently the server keeps the partial text and appends "The mentor's response was cut short — please try again if it looks incomplete." A truncated answer with an explanation is much better than one that stops mid-word.

Embedding service down: retrieval throws, it's caught and logged as `rag.context.error`, and the chat proceeds with no verified-data block. The grounding rules then require the model to label everything as general guidance. Degraded, not broken.

Redis down: the error is logged and the limiter falls through to the in-memory map — permissive rather than blocking, on the judgement that a brief over-allowance is better than taking chat offline.

JUDGEMENT

What would you do differently if you started over?

Four things.

Start with the functional-core pattern everywhere. The roadmap engine is the best-tested code in the project because it's pure — logic separated from I/O, testable without mocks. The chat route is the opposite: SSE parsing, retrieval, persistence and timeout handling all in one handler. Same discipline applied from day one would have made the whole system easier to test.

Type the deadline properly from the start — keep the free-text column for display, add an indexed generated date column for filtering. Retrofitting that now means touching every read path.

Treat the embedding model as a versioned dependency. The threshold bug happened because a model swap silently invalidated a tuned constant. I'd store the model name and dimension alongside every vector, and make a mismatch loud.

Keep architecture docs in CI or delete them. `ARCHITECTURE.md` still describes Supabase and Vercel, two migrations out of date. Documentation that isn't verified rots into something actively misleading. Either the README stays the single source of truth, or a test asserts the doc matches reality.

HONESTY

How much of this did you write versus AI?

Answer this directly and without apology — it's a normal 2026 question, and evasion is the only wrong answer.

Something like: *"I used AI assistance heavily, the way I'd use any modern tooling. What I own is the architecture and every judgement call in it — why retrieval is separated from fact-hydration, why the roadmap engine is pure and how that's enforced, why the CV models are raced rather than chained, why the rate limiter has three windows instead of one. I can walk through any file and explain why it's shaped that way, and I can tell you what I'd change. The bugs I'm proudest of finding — the threshold mismatch and the anonymous chunks — were found by reading production logs, not by a model."*

Then invite the drill-down: *"Happy to pick any subsystem and go as deep as you want."* Confidence plus a genuine offer closes this topic quickly. And it's true — which is exactly why the rest of this document exists.

---

13 Known weak points

A good interviewer will find at least one of these. Naming it before they do converts a gotcha into evidence of self-awareness — every one of these has a defensible reason *and* a fix you can describe.

FIRE-AND-FORGET MESSAGE WRITES

Chat message inserts aren't awaited — `sql`INSERT …`.catch(console.error)`. **Why:** awaiting a write before streaming adds latency to every turn. **Cost:** a failed write silently loses a message from history. **Fix:** await it but off the critical path, or push to a queue with retry.

NON-TRANSACTIONAL RE-INGEST

Re-ingest embeds everything, then deletes old chunks, then inserts new ones in a loop — the code comment says "atomically enough". **Mitigation already in place:** the delete happens only after every embedding succeeds, so an API failure leaves the old vectors intact. **Remaining gap:** a crash between delete and the final insert leaves partial chunks. **Fix:** wrap in a transaction, or insert to a staging table and swap.

BLOCKED REQUESTS STILL CONSUME QUOTA

All three rate-limit counters increment before the decision is made, so a request rejected by the hourly window still burns daily and global quota. **Why:** one atomic Lua round-trip per window is simpler and faster than check-then-increment. **Cost:** a client retrying in a loop drains its daily allowance against 429s. **Fix:** evaluate in a single script that only increments when every window passes.

IN-MEMORY LIMITER IS SINGLE-PROCESS

The fallback `Map` is per-container. Correct for local dev and one instance, incorrect the moment the app scales horizontally. **Say it as:** "Redis is what makes the limit real; the map is a dev convenience and I'd fail loudly rather than silently if Redis were missing in production."

NO EMBEDDING CACHE

Every chat turn pays for an embedding, including repeated questions. **Fix:** hash the normalised query, cache the vector in Redis with a TTL. Cheap, and the single highest-leverage optimisation available.

STALE ARCHITECTURE DOCUMENTATION

`ARCHITECTURE.md` still describes Supabase, RLS and Vercel — accurate as of May 2026, two migrations out of date. The README is current. **Get ahead of this:** if they open that file, say so first. Better still, fix it before the interview.

THE CHAT ROUTE IS LONG

517 lines handling validation, auth, rate limiting, retrieval, prompt assembly, streaming and persistence. **Defence:** it's a linear pipeline with each stage commented, and the genuinely reusable parts are already extracted into `lib/`. **Fix:** pull the SSE parse loop out as a pure function over a string iterator — that's also what would make it unit-testable.

THINGS THAT LOOK LIKE FLAWS BUT AREN'T — DEFEND THESE

Free-text `deadline`: real scholarships say "Rolling" and "TBA". Correct now; add an indexed generated date column for filtering.

No ORM: raw parameterised SQL through Neon's tagged templates. For a schema this size, an ORM adds a query-generation layer between you and the plan, and pgvector operations need raw SQL anyway.

CSS Modules, no UI framework: a deliberate bundle-size and control decision for users on mobile data in Bangladesh.

Empty `apps/api` NestJS scaffold: an abandoned early plan. Say so. Deleting it before the interview is a two-second improvement.

14 If you remember only five things

Under pressure, these five carry the interview. Each one is a mechanism plus the reason it exists.

1 **Retrieval picks relevance; the database supplies truth.**

Vector search returns scholarship IDs. Every fact the model may quote is re-read from the live Postgres row in the same request, because embedded text is a snapshot and a wrong deadline makes a student miss an application.

2 **Every chunk carries its own identity.**

Chunk 0 is a complete fact sheet; every body chunk is prefixed with title, country and deadline. Without it, retrieval returned anonymous fragments and the model attached them to the wrong scholarship.

3 **The roadmap engine is pure, and the purity is enforced by a test.**

No I/O, no clock — the route reads `Date.now()` once and passes it in. A test greps each engine file's source for `fetch(`, `Date.now()` and database imports. A status write cannot move the score, because the scorer's signature doesn't accept one.

4 **Three timeouts, three windows, two model chains.**

15s connect / 45s stall / 90s wall-clock. Hourly / daily / global breaker, first failure wins. DeepSeek → Gemini on separate infrastructure. Every degradation has a user-visible explanation rather than a silent failure.

5 **Both hard bugs were silent, not loud.**

A stale similarity threshold that made retrieval return nothing, and untagged chunks that made the model misattribute facts. Neither threw an exception — both produced plausible wrong answers. That's why the degradation paths have their own log lines.

NUMBERS TO HAVE ON THE TIP OF YOUR TONGUE

AREA	FIGURES
Scale	1.76K Android installs, 4.7★ from 76 reviews · 4,135 web visitors and 17,983 page views per quarter
Code	~33K lines web, ~4.7K mobile · 42 API routes · 254 commits since May 2026
Tests	375 across 18 files, property-based via fast-check
Vectors	1024-d (sliced from 2048) · HNSW cosine · threshold 0.35 · top 10 → 5 scholarships
Chunking	800-char bodies, paragraph/sentence boundaries, one-unit overlap
Rate limits	anon 3/3 · user 6/15 · admin 50/200 · global 20,000/day
Timeouts	15s connect · 45s stall · 90s wall clock · CV race 22s ceiling
Roadmap	6 pillars summing to 100 · confidence floor 40 · PhD research weight 30
Migration	132 users, 54 scholarships, 152 embedding docs, 82 chat sessions — zero downtime

TWO CLOSING HABITS

Lead with the problem, not the technology. "Students were getting wrong deadlines" lands harder than "I implemented RAG with pgvector". The stack is the answer to a question — state the question first.

When you don't know, say so and reason out loud. "I haven't measured that — my guess is X, and I'd check it by Y" is a better answer than a confident wrong one, and interviewers are largely testing how you behave at the edge of your knowledge.

Compiled from the BairePorbo codebase on 29 September 2026 — `main` at `65508e3`. Every figure was read from source or produced by running the test suite. Where the repository's own documentation disagrees with the code, the code is what this brief describes.